MCU – SoC – FPGA

Trade-offs in Performance, Cost and Flexibility

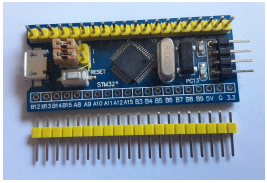
CO3053 Embedded Systems · Semester 261 · Topic 09

The Question

When should you choose an **MCU**, an **FPGA**, or a **SoC**?

- Evaluated across three core dimensions: **performance** (throughput, latency), **cost** (chip, board, power), and **flexibility** (OS support, reconfigurability).
- *Not* aimed at crowning a “best” platform, avoiding the biased goal warning from performance evaluation methodology (`pe2`).
- Every number carries an explicit provenance tag (`pe3`):
  [measured] [simulation] [tool estimate] [analytical] [source]

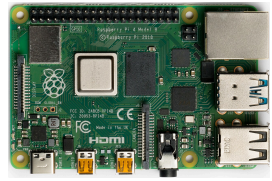
Three Architectures: Root of the Trade-offs



Popolon, CC BY-SA 4.0



S03311251, CC BY-SA 4.0



Laserlicht, CC BY-SA 4.0

MCU

Small sequential core (Cortex-M), on-chip flash & SRAM
No MMU, cannot run Linux
Boot in milliseconds, interrupt < 1 µs
Nanoamp standby power, a few USD

FPGA

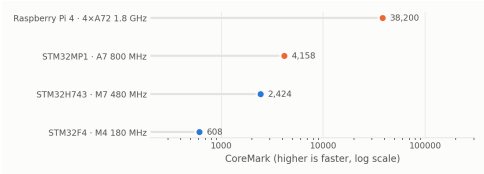
Parallel logic array (LUTs, DSPs, BRAMs wired arbitrarily)
Algorithm mapped to space: **1 sample/cycle**
Deterministic nanosecond latency
Trade-off: higher cost, HDL effort

SoC

Large CPU cores (Cortex-A) with MMU, cache, GPU, codecs
Runs **Full Linux**: networking, USB, AI stacks
Trade-off: non-deterministic latency
Idle power in Watts

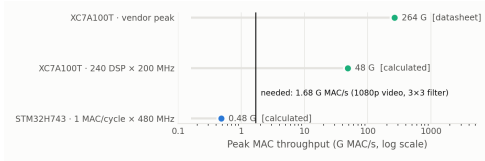
When you need both a rich OS and hard timing: integrate CPU + FPGA on a single die (**Zynq hybrid option**)

Compute Capacity: Sequential CPU vs. Parallel Data Streams



Data: ST overview, OpenBenchmarking (Pi 4), Redrawn

Sequential CPU: SoC Pi 4 is ≈**16×** stronger than MCU H743. FPGA does not run sequential code.

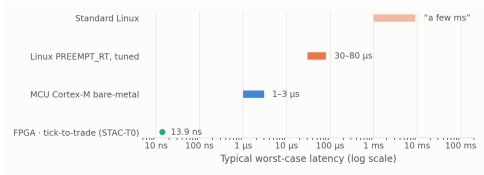


Data: Lee & Seshia Sec 8.1; XA Artix-7 datasheet, Redrawn

1080p video filter: Top MCU meets only ≈29% demand; FPGA uses ≈3.5% DSPs for 100% throughput.

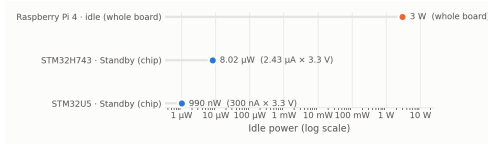
Arucu & Iliev (2025): FIR on FPGA is ≈**27×** faster than Cortex-A9 and uses ≈**95–190×** less energy [analytical].

Worst-Case Latency and Idle Power in the Literature



Data: AC6 (MCU Linux); A-Team Insight, STAC-T0 (FPGA). Redrawn.

Worst-case latency: Separated by 1–2 orders of magnitude (FPGA: ns → MCU: µs → Linux RT: 30–80 µs → standard Linux: ms).



Data: ST H743, Arrow (U5), TechFeed (Pi 4). Redrawn.

Idle power: MCU Standby 300 nA vs SoC 3 W (Δ ≈**3 million times**). Price: MCU £9–12 · SoC $35 · FPGA €144.

Hybrid CPU + FPGA: ADAS and Industrial Control

- Applications requiring **both** a rich OS stack **and** hard real-time timing or high-throughput DSP.
- **Zynq UltraScale+:** combines 4×A53 (Linux), 2×R5 lockstep (real-time safety), and an FPGA fabric on a single chip.
- **Lecture 3, Option #2:** CPU handles soft planning; FPGA guarantees deterministic actuator timing even under CPU load.
- FIR on SoC FPGA+ARM is **8.86×** faster than standalone ARM with superior cost-performance [source].



S03311251, CC BY-SA 4.0

Zynq UltraScale+: Arm + FPGA on one die

Trade-off: higher silicon and tooling cost, lower software flexibility than a plain CPU.

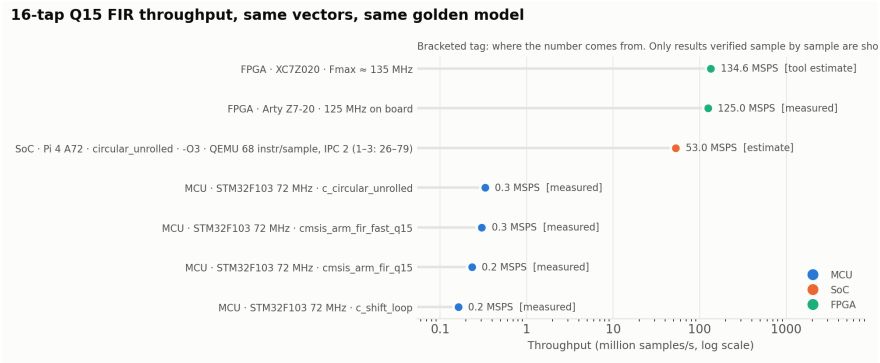
Experiment: One Kernel, Three Platforms

FIR 16-tap Q15, 1024 samples: step, impulse, edge values, saturation stress, random seed 3053
All platforms must match **every single sample (zero discrepancy)** against the Python Golden Model

Platform	Hardware	Empirical Measurement Method
MCU	STM32F103C6 @ 72 MHz (Cortex-M3)	Measured: 4 C/CMSIS variants, DWT cycle counter; TIM2 IRQ under 3 load scenarios
FPGA	Arty Z7-20 @ 125 MHz (XC7Z020 PL)	Measured: 7-stage pure-LUT pipeline (0 DSP); on-chip self-test via JTAG
SoC	Cortex-A72 @ 1.8 GHz (QEMU ARMv8)	Simulated: QEMU AArch64 instruction counter, throughput at IPC 1–3; latency from AC6

Cortex-M3 lacks SIMD instructions (SMLAD), preventing CMSIS-DSP from SIMD acceleration available on M4/M7.

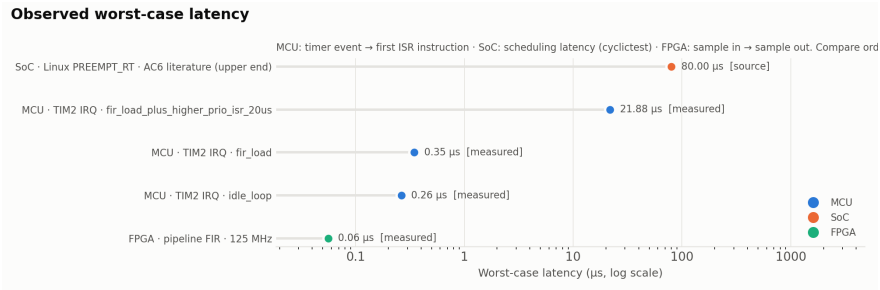
Results: Throughput — FPGA Outperforms by 375×



FPGA: **125.0 MSPS** (1.00 sample/cycle @ 125 MHz) [measured] **SoC**: 53.0 MSPS [estimate; IPC 1–3: 26–79] **MCU**: **0.33 MSPS** [measured]

Technical finding: **FPGA** is **375× faster** than MCU's best variant (and 763× faster than basic C loop). Due to lack of SIMD, MCU achieves only **7%** of theoretical 1 MAC/cycle.

Results: Worst-Case Latency and Determinism (Zero Jitter)



FPGA pipeline: **56 ns** (7 cycles @ 125 MHz), Jitter = **0.00 ns** [measured]
MCU idle IRQ: 264 ns · under FIR load: 264–347 ns [measured]
SoC standard Linux: a few ms [source: AC6] · PREEMPT_RT: 30–80 µs [source: AC6]

Counter-intuitive finding: With a higher-priority 20 µs nested IRQ, MCU worst-case spikes to **21.9 µs**. MCU worst-case is governed by **software architecture**, not raw hardware.

Special Experiment: FPGA Pure Soft-Logic (0 DSPs)

- **Real-world question:** *"If DSP48E1 slices are fully occupied by other tasks, can the FPGA still filter?"*
- Configured RTL for pure soft-logic: `(* use_dsp = "no" *)`, synthesized with Vivado `-max_dsp 0`.
- 16 parallel multipliers and 4-level adder tree mapped onto **Slice LUTs + CARRY4**.
- Safe timing closure: WNS $= +$**0**.**326** ns @ 125 MHz ($F_{\text{max}} \approx 134.6$ MHz).
- Total power: **0.124 W** ($\approx$ **0**.**99** nJ/sample), even lower than DSP version (0.129 W).

Resource	Used	Utilization (%)
DSP48E1	**0 / 220**	**0.00%**
Slice LUTs	**1,672 / 53 200**	**3.14%**
Flip-Flops	**1,046 / 106 400**	**0.98%**
Block RAM	2 / 140	1.43%

Significance: Only **3.1% general logic** on XC7Z020 delivers 125 MSPS, reserving all 220 DSP slices for other pipelines.

Cross-Check Against the Literature

Verification Question	Our Measurement	Literature Claim
How close does MCU get to “1 MAC/cycle”?	**7%** (216.3 cycles/sample)	PLAN assumed 100% (≈0.48 G MAC/s on H743)
How much faster is FPGA than MCU / CPU?	**375×** (vs MCU); 2.4× (vs Pi 4)	≈27× over Cortex-A9 (Arucu & Iliev)
MCU bare-metal interrupt latency	264–347 ns (idle/load)	1–3 µs (AC6)
MCU worst-case with nested 20 µs IRQ	**21.9 µs**	Software design governs worst-case
What happens when scaled to 64 taps?	MCU throughput drops 4× (80 kS/s)	FPGA retains 125 MSPS (gap widens to >1 500×)

Secondary observation: An x86 desktop core filters 140 MSPS. FPGA does not win on raw speed on small filters, but on **constant throughput with scaling**, **fixed latency**, and **low power**.

Application Map: Where Each Platform Wins

MCU: Automotive ECU, Motor, Battery Sensors
- AURIX lockstep certified for **ASIL-D**, AEC-Q100 150 °C
- Millisecond boot, sub-µs IRQ, a few USD
- *Why chosen:* SoC is slow to boot & jittery; FPGA is costly.

SoC: Wi-Fi Routers, Smartphones, IP Cameras
- Qualcomm IPQ: Linux + 37.5 Mpps packet engine
- Snapdragon: CPU + GPU + NPU + 5G modem on 3 nm
- *Why chosen:* MCU lacks MMU; FPGA HDL is too costly.

FPGA: Ultrasound, 5G Radio, ASIC Emulation, HFT
- Ultrasound 128–1024 channel ADC beamforming
- Reconfigurable 5G O-RU; HFT tick-to-trade 13.9 ns
- *Why chosen:* MCU lacks throughput; SoC lacks determinism.

Hybrid: Autonomous ADAS, Industrial Robotics
- Zynq: Cortex-A (Linux/AI) + FPGA (hard actuator timing)
- *Why chosen:* No single platform satisfies both constraints.

Platform Decision Matrix

If your application requires…	Choose	Because
Multi-year battery life, light tasks	**MCU**	300 nA Standby; >10 years on CR2032
Tens of µs deadline, safety certified	**MCU**	Sub-µs interrupt; ISO 26262 ASIL-D
Linux, camera, networking, GUI; soft timing	**SoC**	CoreMark ≈16× H743; drivers available
Continuous >0.5 G MAC/s, streaming I/O	**FPGA** / Hybrid	1 sample/cycle (E2), ≈44–48 G MAC/s
Fixed nanosecond latency	**FPGA**	Pipeline 56 ns (0 jitter), HFT 13.9 ns
Both rich OS and hard real-time timing	Hybrid	Zynq, Lecture 3 Option #2

Core design rule: Demanding higher throughput and determinism requires paying with **higher BOM cost** and **lower software flexibility**.

“There is no best platform,

only the platform that is **SUFFICIENT** for the deciding metric, at the lowest cost.”

— Core design insight

MCU	Wins on **cost**, **idle power** (nA), and **safety certification**. Limited to sequential computation.
SoC	Wins when a **Linux OS**, networking, and rich media are needed. Limited by non-deterministic latency.
FPGA	Wins on **parallel throughput** and **nanosecond latency**. Limited by BOM cost and HDL effort.

Thank You

Questions & Discussion

CO3053 Embedded Systems · Semester 261 · Topic 09

Appendix: Anticipated Questions

- **“Is the MCU code fully optimized? Why not Cortex-M4/M7?”** We benchmarked CMSIS-DSP. M3 represents cost-sensitive MCUs lacking SIMD. Cortex-M4/M7 SIMD (`SMLAD`) is 3–4× faster (50 cycles/sample), but still hundreds of times slower than FPGA which computes all 16 taps in parallel.
- **“Is FPGA jitter truly 0 ns?”** Yes at the cycle-accurate logic level: exactly 7 cycles latency. Physical signal-level jitter is merely quartz phase jitter (picoseconds), negligible compared to MCU/SoC microsecond variance.
- **“How was FPGA power measured?”** Vivado `report_power` (vector-less) tagged as [tool estimate], including 0.106 W device static power.
- **“What happens if scaled to 64 taps?”** MCU throughput drops 4× (80 kS/s). FPGA adder tree expands by 2 stages (latency rises to 9 cycles = 72 ns), but **throughput remains exactly 125 MSPS**. The performance gap widens from 375× to >1 500×.

Appendix: Main Sources & Image Credits

Main References:

- ST: STM32H7/F4/MP1 overview, STM32H743; OpenBenchmarking: Raspberry Pi 4 CoreMark
- Arucu & Iliev, J. Low Power Electron. Appl., 2025 (FIR on FPGA, GPU, CPU)
- AC6: PREEMPT_RT real-time latency; A-Team Insight: Exegy/AMD STAC-T0 (13.9 ns tick-to-trade)
- Embedded World 02/2026; Automotive World (AURIX TC3xx); Future Electronics (NXP S32K)
- Macnica, CNX Software (Qualcomm IPQ); GSMArena (Snapdragon 8 Elite)
- Kou et al., 2023 (Ultrasound beamforming on FPGA); Synopsys (HAPS ASIC prototyping)
- Lee & Seshia, *Introduction to Embedded Systems*, Sec 8.1–8.2; Lecture 2, Lecture 3.

Images: Wikimedia Commons (Creative Commons open licenses indicated per image). Charts redrawn from published data.